

PREGUNTAS DE PARCIALES

1.Un equipo de desarrollo de tres integrantes se encuentra trabajando en un proyecto ágil. Detalle qué son las daily y cómo podrían realizarlas para que sean eficaces y sigan las buenas prácticas. Realice lo mismo para las retrospectivas.

Las daily son reuniones que duran 15 minutos aproximadamente, en este participan los desarrolladores y el product owner junto al scrum master pueden participar como oyentes. En esta se realizan preguntas clave: ¿qué hice ayer? ¿Qué haré hoy? ¿Tuve alguna dificultad o conflicto? Esto ayuda a que todos los desarrolladores entiendan el estado del trabajo en el transcurso de los días y puedan definir problemas tempranamente. Suele ser mientras todos los integrantes están parados así se promueve la agilidad y es eficaz. Una buena manera de realizar una buena daily es no hacer preguntas específicas sobre el código, ya que al ser las reuniones de poco tiempo, es mejor priorizar el mismo en cuestiones más generales del proyecto.

En las retrospectivas participan los integrantes del scrum: desarrolladores, scrum master, product owner. En esta se analizan los errores, dificultades y también las cosas que salieron bien. Luego de este análisis se determina como se puede mejorar según los resultados del sprint terminado y se implementan para el siguiente. Lograr una buena Retro se basa en no insultar ni decir nombres en particular ya que puede romper con la dinámica del equipo. Sin embargo, sí son aceptadas las críticas constructivas hacia dinámicas de trabajo que no son del todo correctas.

2.Dado el siguiente contexto:

Se desea desarrollar una aplicación para la carga de datos de inspecciones de automotores para una empresa de seguros.

La empresa posee una dotación de 100 agentes inspectores. Cada agente de seguros, portador de un teléfono o notebook según se defina y que la empresa le entregará, recibirá a través de la aplicación la agenda del día con los datos de los automotores a inspeccionar. Cada agenda está identificada con el número de empleado del agente el cual consta de veintiún caracteres alfanuméricos; este será verificado al momento de ser recibida.

Las agendas son diarias y cada una no podrá incluir más de 6 inspecciones. Además, la aplicación no podrá recibir una agenda si previamente no envió a la empresa todas las inspecciones de la agenda del día anterior.

El agente se traslada al domicilio del propietario y realiza la inspección del automotor agendado para ser asegurado.

Al realizar cada inspección, el agente deberá registrar la marca, el modelo y los números de serie de cada componente (chasis, ruedas, motor, carrocería) y verificar y registrar su estado de conservación. Cada componente podrá estar en buen, regular o mal estado. En el caso de no poder ser identificado deberá marcarse como “no legible”.

Las inspecciones podrán realizarse de una vez o podrán empezarse y continuarse para ser terminadas en otro momento.

Las inspecciones podrán ser canceladas y también marcadas como de imposible realización, registrando la razón.

Una vez completada una inspección, ésta podrá ser enviada a la empresa o almacenada para ser enviada a posteriori. Las inspecciones no realizadas o canceladas no podrán ser enviadas a la empresa.

Se pide:

- 1. Elegir 3 escenarios principales y escribirlos en el formato de historia de usuario.**
- 2. Pensar 3 atributos de calidad que le parezcan relevantes, detallarlos y especificar cómo medirlos.**
- 3. Armar un modelo de dominio preliminar.**

1. Historias de usuario, 3 escenarios principales:

Título: recepción de agenda diaria	Prioridad: X	Estimación: X
Como: Agente inspector Quiero: Recibir mi agenda diaria con las inspecciones asignadas Para: Poder conocer los automotores a inspeccionar ese día		
Dado que: ya envíe todas las inspecciones del día anterior cuando: la aplicación solicite la agenda del día entonces: debería recibir la nueva agenda con hasta 6 inspecciones Dado que: no envíe todas las inspecciones el día anterior cuando: intente recibir la agenda del nuevo día entonces: la aplicación debe rechazar la recepción y enviar un error Dado que: se recibe una agenda cuando: se valide el número de empleados entonces: solo se aceptará si el formato es correcto (21 caracteres alfanuméricos)		

Título: Registro de inspección	Prioridad: X	Estimación: X
Como: agente inspector Quiero: registrar los datos y el estado de cada componente del automotor Para: poder documentar la inspección asignada de forma completa y precisa		
Dado que: el agente inicia una inspección cuando: cargue la marca, modelo y número de serie de los componentes entonces: el sistema deberá permitir guardar los datos Dado que: una inspección está en progreso cuando: el agente decida interrumpirla entonces: podrá guardar el avance para continuarla más tarde Dado que: un componente no puede identificarse cuando: el agente lo marque como "no legible" entonces: el sistema deberá registrar el estado		

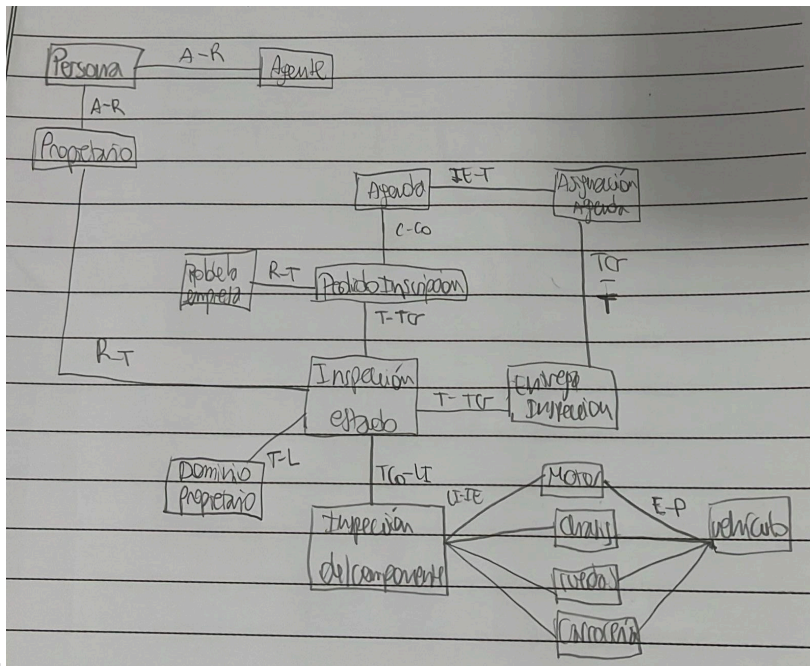
Dado que: una inspección no puede realizarse o se cancela
cuando: el agente seleccione esa opción y justifique la razón
entonces: el sistema deberá registrar el motivo y marcar su nuevo estado

Título: envío de inspecciones completadas	Prioridad: X	Estimación: X
Como: agente inspector quiero: enviar a la empresa las inspecciones completadas para: mantener actualizada la información y habilitar la recepción de nuevas agendas		
Dado que: una inspección está completada cuando: el agente elija enviarla entonces: la aplicación debe transmitirla correctamente a la empresa Dado que: el agente no tiene conexión cuando: intente enviar la inspección entonces: la aplicación deberá permitir guardarla localmente para luego enviarla Dado que: una inspección está cancelada o no realizada cuando: el agente intente enviarla entonces: el sistema deberá impedir el envío y mostrar una advertencia		

2. Atributos de calidad a considerar: escalabilidad, capacidad que tiene un sistema para manejar la carga del trabajo sin afectar el rendimiento de esta. Trata de poder manejar a todos los usuarios que se conectan, aunque sea una gran cantidad. Se mide con el número máximo de usuarios concurrentes o solicitudes por segundo atendiendo sin superar un umbral de tiempo de respuesta.

Disponibilidad, porcentaje del tiempo en que el sistema está accesible y funcionando correctamente para los agentes, especialmente durante su horario laboral. Porcentaje de *uptime* en un período determinado (por ejemplo, 99,5% mensual).

Seguridad, que es la capacidad del sistema para mantener la información completa y correcta, sin pérdida o corrupción de datos. Se puede medir con el porcentaje de transacciones completadas sin errores o pérdidas de datos.



3.

3. Le piden hacer una revisión de este código que forma parte de un proyecto en el que participa.

```

public class Lamp {
    private int brightness; // Intensidad de la lámpara (0 a 100)
    private boolean isOn; // Estado de la lámpara
    private int timer; // Temporizador en minutos

    public Lamp() {
        this.brightness = 50; // Valor predeterminado de brillo (50%)
        this.isOn = false;
        this.timer = 0; // Sin temporizador por defecto
    }

    public void turnOn() {
        if (!isOn) {
            this.start();
            isOn = true;
        }
    }

    public void turnOff() {
        if (isOn) {
            this.stop();
            isOn = false;
        }
    }

    public void setBrightness(int b) {
        if (b <= 0) this.brightness = 0;
        else if (b >= 100) this.brightness = 100;
        else this.brightness = b;
    }

    // public void setTimer(int m) {
    //     if (m > 0) {
    //         this.timer = m;
    //     }
    // }

    // public void CheckTimer() {
    //     if (timer > 0) {
    //         timer--;
    //     }
    //     if (timer == 0)
    // }
}

public class Button {
    private Lamp lamp;

    public Button() {
        this.lamp = new Lamp();
    }

    public void press() {
        lamp.turnOn();
    }

    public void pressWithBrightness(int brightness) {
        lamp.turnOn();
        lamp.setBrightness(brightness);
    }
}

public class Main {

    public static void main(String[] args) {
        Button button = new Button();

        button.press(); // enciende la lámpara
        button.press(); // apaga la lámpara
        button.pressWithBrightness(75); // Enciende la lámpara y ajusta el brillo a 75%
        button.press(); // apaga la lámpara
    }
}

```

1. Identifique problemas y malas prácticas utilizando los conceptos, terminologías y temas vistos en la materia.

Al analizar el código se observan varias malas prácticas de diseño orientado a objetos. En primer lugar, existe un **alto acoplamiento** entre las clases `Button` y `Lamp`, ya que el botón crea internamente la instancia de la lámpara. Esto provoca que el botón dependa directamente de la implementación concreta de la lámpara, dificultando su reutilización y violando el principio de inversión de dependencias.

Además, la clase `Button` asume responsabilidades que no le corresponden, como decidir el nivel de brillo de la lámpara. Esto constituye una violación del **Principio de Responsabilidad Única**, dado que el botón debería limitarse a accionar la lámpara y no a configurar su estado interno.

Por otro lado, la clase `Lamp` presenta problemas de **encapsulamiento**, ya que no expone claramente métodos para consultar o modificar su estado de forma controlada. También se observa la presencia de métodos confusos, como `turnOn`, que realiza acciones poco claras y hace referencia a comportamientos que no están correctamente definidos dentro de la clase.

Asimismo, el código contiene secciones comentadas que no se utilizan, lo cual se considera una mala práctica, ya que introduce ruido y dificulta la lectura y el mantenimiento del sistema. Finalmente, la clase `Main` depende de detalles internos del funcionamiento de los objetos, lo que evidencia un bajo nivel de abstracción y un diseño poco flexible.

En conjunto, estos problemas afectan la legibilidad, mantenibilidad y extensibilidad del sistema, y muestran un uso inadecuado de los principios fundamentales de la programación orientada a objetos.

2. Reescriba el código para mejorarlo.

```
public class Lamp {  
  
    private int brightness;  
    private boolean isOn;  
  
    public Lamp() {  
        this.brightness = 50; // brillo por defecto  
        this.isOn = false;  
    }  
  
    public void turnOn() {  
        isOn = true;  
    }  
  
    public void turnOff() {  
        isOn = false;  
    }  
}
```

```

    }

    public void setBrightness(int brightness) {
        if (brightness < 0) {
            this.brightness = 0;
        } else if (brightness > 100) {
            this.brightness = 100;
        } else {
            this.brightness = brightness;
        }
    }

    public boolean isOn() {
        return isOn;
    }
}

public class Button {

    private Lamp lamp;

    public Button(Lamp lamp) {
        this.lamp = lamp;
    }

    public void press() {
        if (lamp.isOn()) {
            lamp.turnOff();
        } else {
            lamp.turnOn();
        }
    }
}

public class Main {

    public static void main(String[] args) {
        Lamp lamp = new Lamp();
        Button button = new Button(lamp);

        button.press();           // enciende
        button.press();           // apaga

        lamp.setBrightness(75);   // se configura la lámpara
        button.press();           // enciende con brillo ajustado
    }
}

```

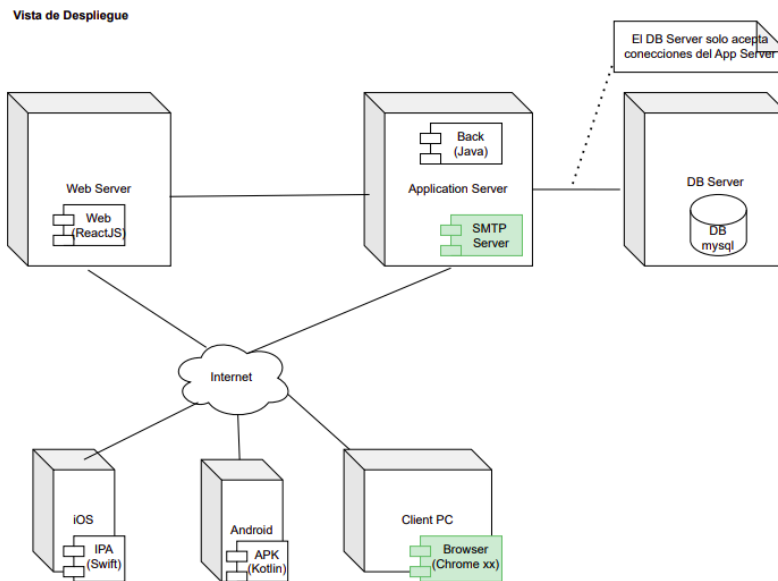
```
}
}
```

4.Explique para qué se utiliza la vista física en el modelo de Vistas de 4+1.

Dé un ejemplo.

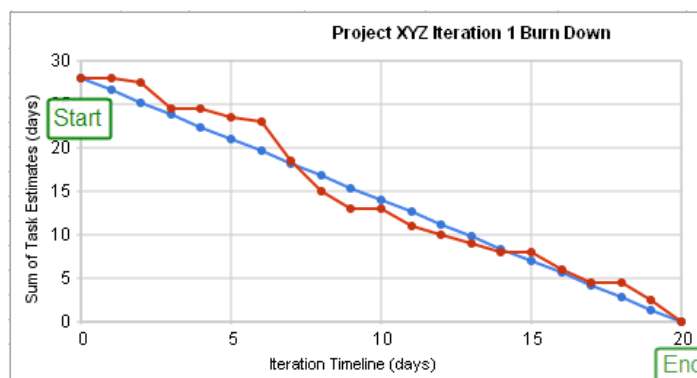
La vista física se enfoca en los requisitos no funcionales como la disponibilidad, el rendimiento, la escalabilidad y confiabilidad. Nos muestra como los componentes físicos se dividen en una red de nodos y cómo se asigna cada elemento a estos nodos.

Configuraciones flexibles para un impacto mínimo en el código fuente.



5.Explique qué muestra un diagrama Burndown Chart. Realice uno de ejemplo y explique qué podría estar pasando en dicho ejemplo.

Un diagrama Burndown chart muestra dos líneas principales, el gráfico tiene en el eje x el tiempo del sprint y en el eje y los storypoints. En este, una línea representa el trabajo avanzando de una forma ideal según las estimaciones hechas para los storypoints, luego en otra línea se representa como fue avanzando el trabajo en tiempo real, es decir, de la forma que lo realizó el equipo, si esta línea sube es porque se dividieron tareas o se hicieron mal.



En la línea azul se puede observar el trabajo avanzando idealmente considerando los story points estimados; en cambio, en la línea roja se muestra el trabajo realizado por el equipo. Podemos decir que no quedan tareas pendientes para el siguiente sprint ya que para el día 20 se tenían todas las tareas completadas.

6. Detalle los síntomas de un diseño en deterioro o mal diseño.

Los síntomas de mal diseño son varios, pero los principales son:

RIGIDEZ: se da cuando modificar una parte del código se vuelve muy difícil, requiere muchas modificaciones extra.

FRAGILIDAD: ocurre cuando al cambiar algo, cambia muchas partes del código, generando varios errores en lugares indeseados e inesperados.

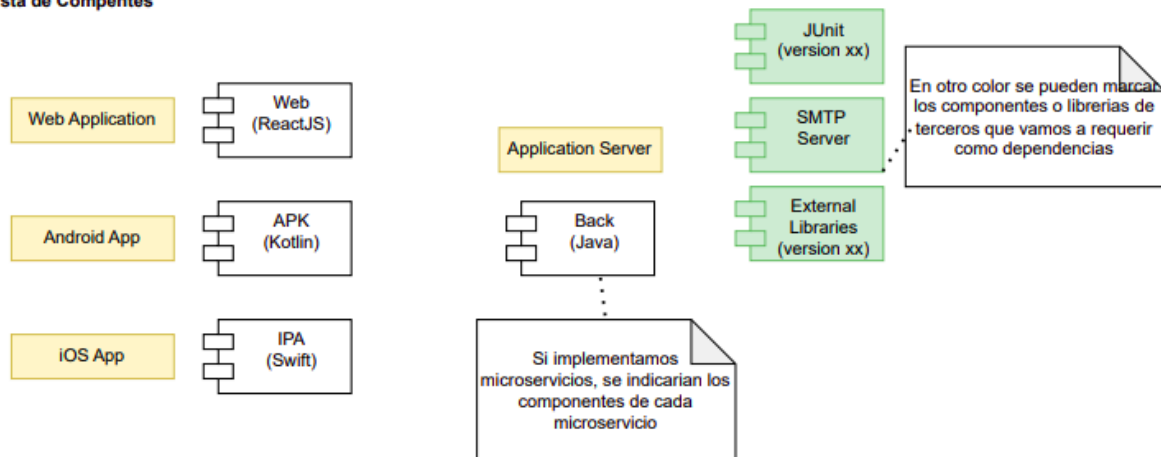
INMOVILIDAD: sucede cuando, por más que queramos reutilizar una parte del código, no podemos hacerlo porque está tan entrelazado con otras cosas que no es posible extraerlo.

VISCOSIDAD: cuando es más fácil hacer algo mal, que hacerlo bien.

1. Explique para qué se utiliza la vista de desarrollo (o componentes) en el modelo de Vistas de 4+1. Dé un ejemplo.

La vista de componentes se enfoca más en el entorno de desarrollo de software y cómo está organizado, ya sea en bibliotecas o módulos. Tiene que ver más con el lenguaje de programación que utilizamos, las herramientas, entre otros. Clave: administración del software.

Vista de Componentes



3. Un equipo de desarrollo de tres integrantes se encuentra trabajando en un proyecto ágil, utilizando la técnica de Poker Planning y utilizando la siguiente secuencia de Fibonacci (1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987...) para asignar peso a las tareas. Al comenzar la estimación de tareas se encuentran con el siguiente conflicto: Uno considera que una tarea debe recibir el peso de 144, sin embargo, los otros dos creen que debe ser de 233, y el otro integrante les responde lo mismo, pero con los valores 3, 5 y 8. Eso les pasa en casi todas las tareas, y les cuesta llegar a consenso porque terminan discutiendo mucho tiempo las estimaciones. ¿Qué les recomendaría?

Les recomendaría seguir los siguientes pasos: primero consultar si todos conocen sobre lo que se está estimando, si entienden sobre los temas, si alguno tiene alguna duda, etc. Luego les recomendaría que se pregunten si todos están de acuerdo en lo que se está estimando: si es el tiempo que lleva la tarea, la dificultad, el esfuerzo, entre otras. También una buena opción es ver la escala que estamos utilizando, por que a veces escalas muy grandes puede dificultar la elección a la hora de estimar. Por último si no hubo mejoras con los puntos anteriores, se podría tomar un promedio de las estimaciones dadas o elegir la que está en el medio ya que es lo que más se asemeja para todas las partes sin quedar desfasado o muy concentrado en una opción.

4. Dado el siguiente contexto: Se desea implementar un cliente de mail.

El mismo debe permitir al usuario registrarse, y luego configurar sus casillas de email. Va a poder ver el contenido de su casilla de mensajes. También enviar emails.

Disponemos de librerías (SMTP y POP) capaces de interactuar con los servidores de email de las casillas para realizar envíos, recibir mensajes y borrarlos.

También vamos a poder guardar contactos, con datos personales asociados a cada dirección de email. Nos interesa contar con un detector de spam que evalúe atributos como: IP de origen, dirección de origen de un contacto, la presencia de ciertas palabras clave, etc. Cada evaluación provee un score por atributo indicando si lo considera spam; con esa información se marcará como spam o no.

Comenzaremos con métodos de evaluación básicos, pero esperamos que la experiencia nos permita eventualmente mejorar el sistema, ya sea agregando nuevas evaluaciones o modificando las actuales. A futuro, se está considerando reusar los mecanismos del detector de spam para detectar categorías de mail, como ser: social, promociones, comerciales, etc.

Se pide:

1. Elegir 3 escenarios principales y escribirlos en el formato de historia de usuario.
2. Pensar 3 atributos de calidad que le parezcan relevantes, detallarlos y especificar cómo medirlos.
3. Armar un modelo de dominio preliminar.

Título: Utilizar las acciones básicas de los mails	Prioridad: X	Estimación: X
Como: usuario que utiliza mails quiero: registrarme en la plataforma de mails para: poder enviar mensajes o recibirlos		
Dado que: el usuario se registra correctamente en la aplicación cuando: intente configurar sus casillas de mail entonces: la página debería poder mostrar el contenido de su casilla Dado que: el usuario escriba un mensaje para enviar cuando: presione el botón para enviar el mensaje entonces: la aplicación deberá enviar el mensaje al destinatario Dado que: el usuario recibe un mensaje cuando: le llegue la notificación del mensaje y entre a este entonces: la página le deberá mostrar el contenido del mensaje que recibió		

Título: Utilizar las acciones básicas de los mails		
Como: usuario que utiliza mail quiero: guardar contactos para: poder tener un acceso más rápido y directo hacia las direcciones de mail de las		

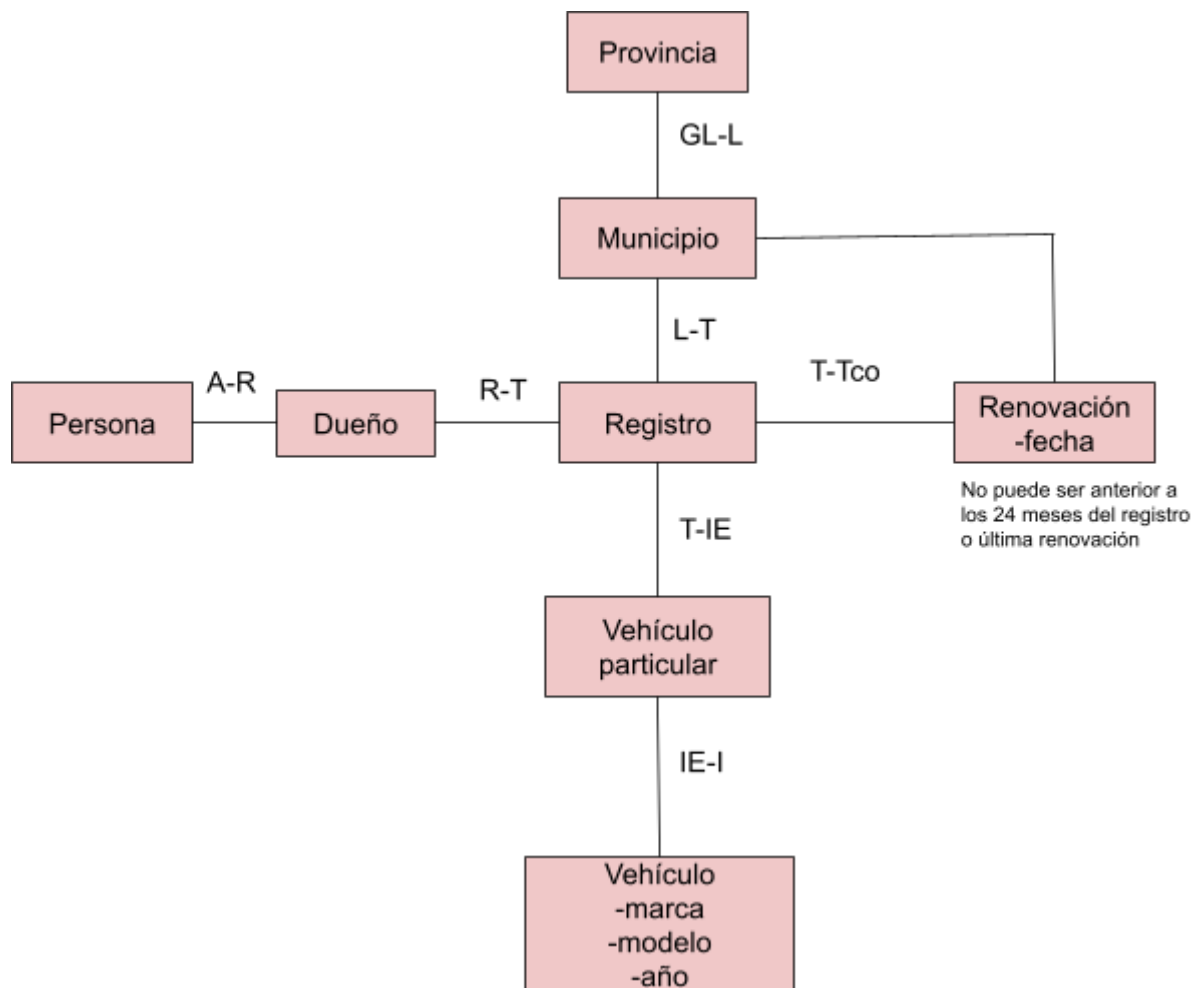
personas más recurrentes
Dado que: el usuario guarda un contacto cuando: intenta enviar un mail a un contacto guardado entonces: el sistema deberá poder identificar ese contacto y enviarle el mail Dado que: el usuario elimina un contacto cuando: el usuario busca sus contactos entonces: el sistema no debería mostrar el contacto recientemente eliminado

Título: Ver mails importantes	Prioridad: X	Estimación: X
Como: usuario que utiliza mails quiero: ver qué mails son importantes para: evitar leer aquellos marcados como spam		
Dado que: el usuario ingresa a su casilla cuando: ve los mails más recientes entonces: la aplicación debería señalar aquellos detectados como spam y no mostrarlos		

Algunos atributos de calidad a tener en cuenta pueden ser:

- Disponibilidad: se refiere a que el sistema esté disponible el mayor tiempo posible cuando se lo necesita. Se mide en porcentaje de tiempo en funcionamiento en relación al tiempo total.
- Performance: corresponde a los tiempos de respuesta en relación con las funcionalidades de la aplicación. Se mide con la latencia, que es el tiempo de respuesta de un evento, y la capacidad, que es el número de eventos que pueden ocurrir en un tiempo determinado.
- Seguridad: es la capacidad que tiene la aplicación de mantener la información completa, correcta y segura. Se puede medir con el porcentaje de transacciones completas o sin pérdidas de datos.

3. La registración de un vehículo se realiza en un municipio de una provincia. Involucra a un vehículo en particular y a su dueño. Cada vehículo tiene una descripción que incluye su marca, modelo y año. Cada registración puede renovarse múltiples veces en el mismo o diferente municipio, y no pueden repetirse antes de los 24 meses. Realice un modelo de dominio del problema preliminar.



5. Una empresa quiere desarrollar una aplicación que “permita a los usuarios comunicarse entre sí”. Como ingeniero/a de software recién contratado/a, te piden que empieces a trabajar en el diseño del sistema.

a) Indique los pasos a seguir.

b) Liste al menos 5 preguntas relevantes que harías al cliente.

Primero, lo más importante es hacerle preguntas al usuario sobre los requerimientos mínimos del sistema. Luego, analizamos esos requerimientos y los separamos en funcionales y no funcionales. Una vez hecho esto, creamos los distintos diagramas y elementos que ayuden a un mejor entendimiento del problema (diagramas UML, historias de usuario, mapas de empatía, User Personas, entre otros), junto con las diferentes vistas de la arquitectura (vistas de arquitectura 4+1).

Una vez terminada la fase del entendimiento del problema, pasamos a la etapa de diseñar el código en sí, junto con la base de datos y cómo se comunicarán los diferentes módulos, es decir, entramos en la fase de prototipado. Así, podremos pasar a la etapa de planificación de tiempos y tareas en sprints y los lenguajes y herramientas a utilizar.

A medida que se avanza con el proyecto, es importante ir creando tests unitarios que vayan comprobando las diferentes funcionalidades del sistema para así lograr un mantenimiento constante del mismo y evitar cualquier uso incorrecto del mismo.

Algunas preguntas que se le pueden hacer al usuario son:

¿Qué tipo de comunicación será? ¿Por chat, videollamada?

¿Qué plataformas deberá soportar?

¿Cuántos clientes esperan tener en simultáneo?

¿Qué medidas de seguridad esperan tener?

¿Desean el uso de notificaciones o estados de usuario, como en línea, escribiendo, etc.?

¿Hay alguna restricción en cuanto a tiempos, presupuesto o tecnologías a usar?